PolNBDNet: Real-Data Training & Wishart-Noise Effectiveness Report

Goal: Pull real UAVSAR SAR data, run it through the PolNBDNet preprocessing pipeline, train the NBDNet despeckling model, and measure how effective the pipeline is at recovering a clean signal from Wishart-simulated speckle, across different noise severities (look counts, L).

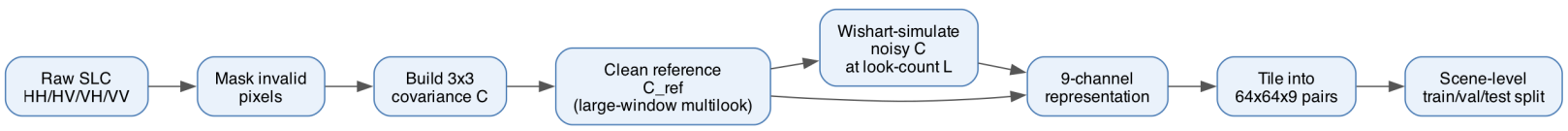
This report is organized week by week — each section below is a self-contained record of that week's work, in the order it happened. Later weeks build on earlier ones but don't rewrite them; where something from an earlier week was later fixed or changed, that's noted inline with a pointer forward.

Week 1 — 2026-10-11: Pipeline + small-scale real-data run

Goal this week: get the preprocessing pipeline running end-to-end on real UAVSAR data (not synthetic), train NBDNet, and get a first read on whether the Wishart-noise/despeckling approach actually works.

1.1 Pipeline recap

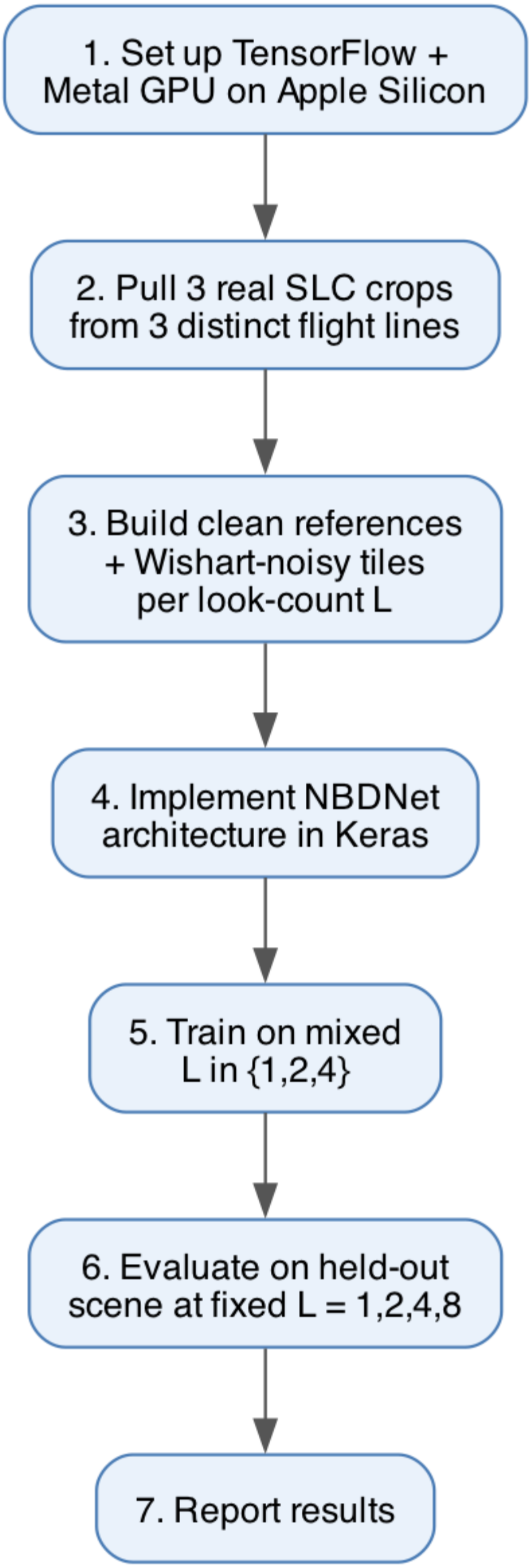
The preprocessing pipeline (built earlier, reused today) implements the 13-step guide for turning raw polarimetric SAR SLC data into despeckling training pairs:



{width=100%}

`C_ref` (step D) stands in for the unobservable "true" speckle-free signal. `simulate_polsar_speckle` (step E) draws correlated noise from the complex Wishart distribution — the polarimetrically-correct way to inject synthetic speckle, preserving cross-channel phase instead of corrupting each channel independently.

1.2 What we did



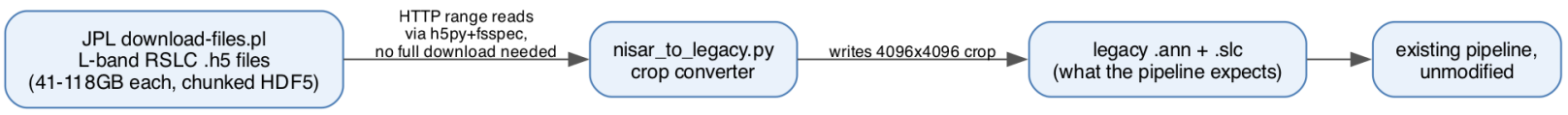
{height=7.5in}

Environment

Apple M4, macOS, 16GB RAM. TensorFlow 2.21 on the default Python 3.13 had no working GPU plugin (`tensorflow-metal` isn't yet compatible with Python 3.13 or TF 2.20+'s internal ABI). Built a dedicated conda env (`polnbdnet` , Python 3.11, TensorFlow 2.16.2 + tensorflow-metal 1.2.0) — confirmed a real matmul executing on the Apple M4 GPU before proceeding.

Data: where it actually came from

ASF (the standard public UAVSAR archive) does not host raw SLC — confirmed earlier by scanning all 12,521 UAVSAR granules there; zero have a `.slc` file. Real raw quad-pol SLC turned out to be available from JPL directly, packaged in **NISAR-format HDF5** (a different container format than the classic UAVSAR flat-binary `.slc` , used because JPL is validating NISAR's ground-processing pipeline against UAVSAR data ahead of the NISAR satellite's launch):



{width=100%}

Because a full flight line is ~1.8 billion pixels (far too large to hold in memory for the dense covariance/Wishart steps), only a 4096×4096 crop was pulled from each of 3 **genuinely distinct real sites** — chosen specifically so no spatial/scene overlap is possible:

Split	Scene	Site
train	`howlnd`	Howland Forest, Maine
val	`bermsr`	BERMS boreal forest, Canada
test	`monda`	Mondah forest, Gabon

Raw crop footprint: ~512MB each (~1.5GB total).

Training data generation

For train/val, each tile got a **random** look-count L ∈ {1, 2, 4} (matching the paper's own training recipe and the guide's recommendation to vary L so the network generalizes across noise severities). For the held-out test scene, the *same* tile grid was re-used to generate **four separate, fixed-L** datasets (L = 1, 2, 4, 8), so recovery quality could be compared apples-to-apples across noise severity.

Split	Scene	Tiles	L
train	howlnd	3,844	random {1,2,4}
val	bermsr	3,844	random {1,2,4}
test	monda	3,844 × 4	fixed 1, 2, 4, 8 each

dB normalization range (`db_min=−19.88` , `db_max=−4.82`) was fit on the **train scene only**, per the guide's explicit warning against fitting normalization stats on val/test data. *(This single global range turned out to be a domain-shift risk — fixed in Week 2, §2.1.)*

1.3 Model architecture

Reconstructed from the paper's Fig. 1–4 (the paper doesn't give numeric layer configs beyond what's drawn, so this is a best-effort faithful implementation), adapted for the pipeline's 9-channel representation instead of the paper's single-channel SAR intensity:

```
\begin{landscape}
```



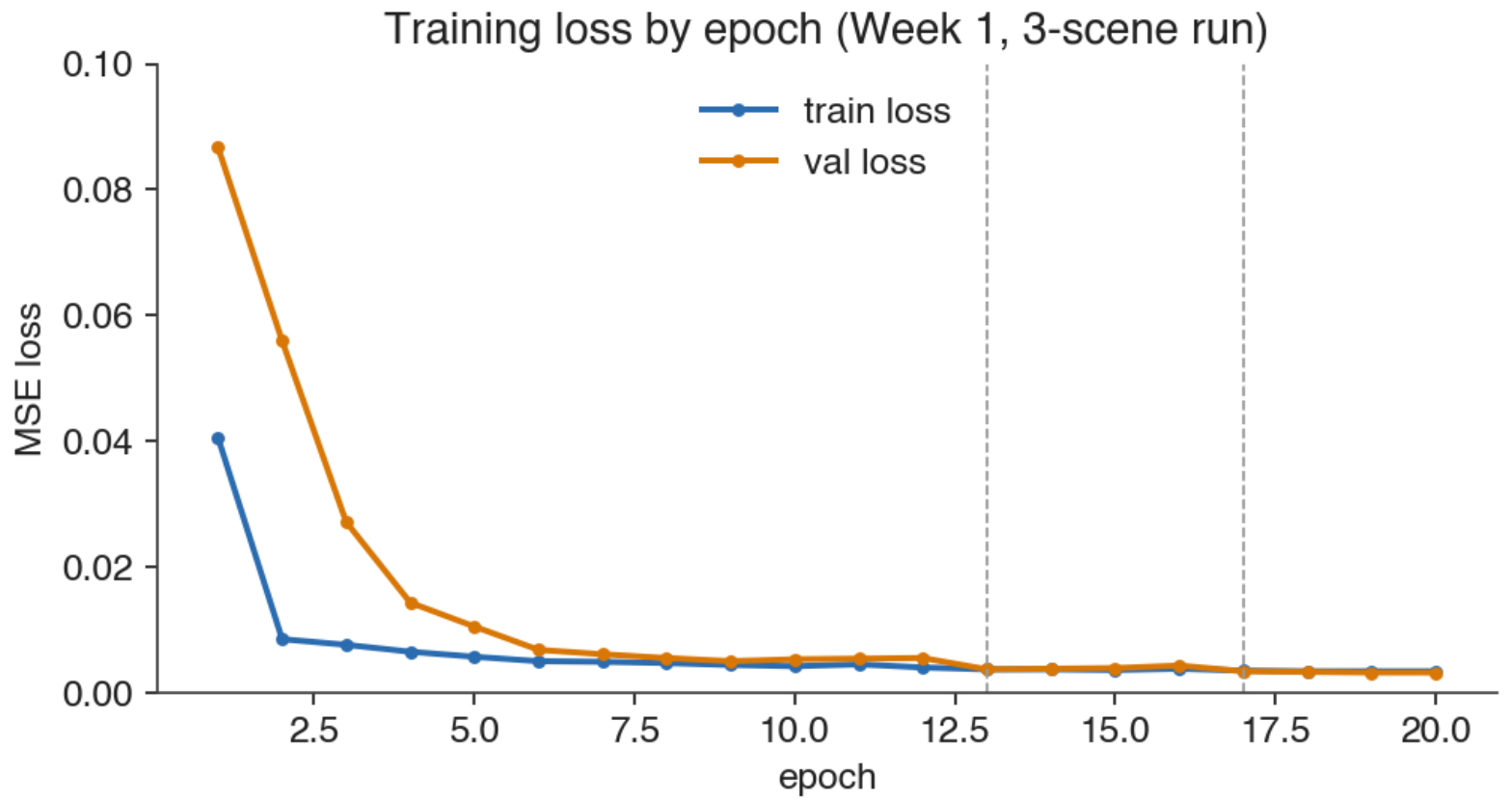
{width=95%}

```
\end{landscape}
```

- **Squeeze-Excitation**: global-pool → FC → FC → sigmoid channel gate
- **CBAM**: channel attention (avg+max pool → shared MLP → sigmoid) followed by spatial attention (channel-pooled avg/max → conv → sigmoid)
- **Skip connection** around the 256-channel bottleneck, matching the paper's described topology
- 1,421,083 parameters total
- Trained with MSE loss between predicted and clean (`C_ref`) 9-channel representations — same loss the paper uses

1.4 Training

Adam optimizer, lr=0.001 (paper's setting), batch size 32, up to 20 epochs with `ReduceLROnPlateau` and `EarlyStopping` (best weights restored). Ran 3,130s (~52 min) on the M4 GPU.

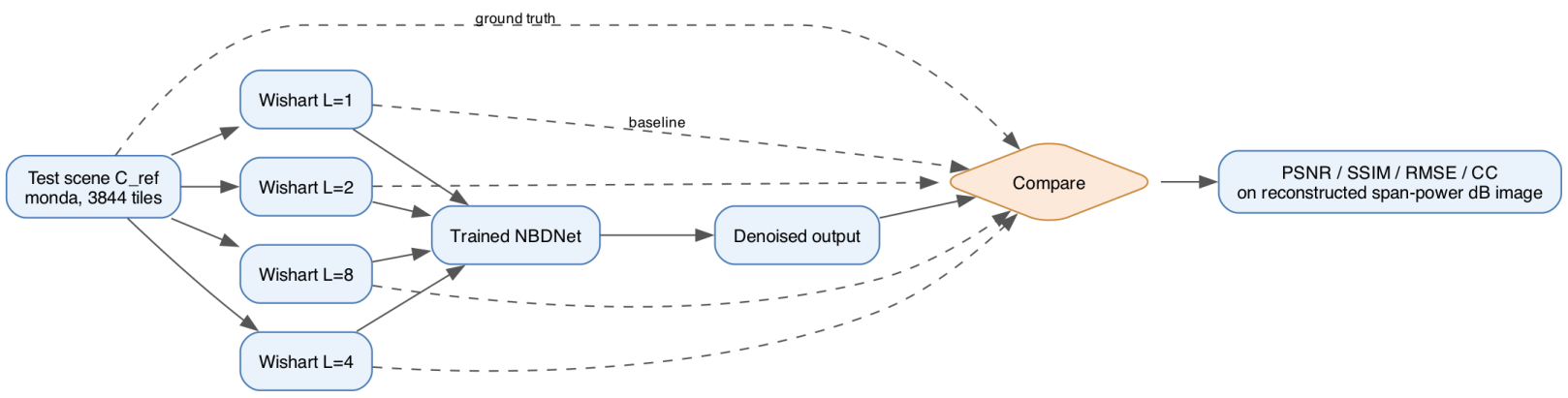


{width=90%}

(dashed lines mark the epochs where learning rate was halved via plateau detection)

Final: train loss 0.0035, val loss 0.0033 — no overfitting (val tracked train closely throughout).

1.5 Evaluation design

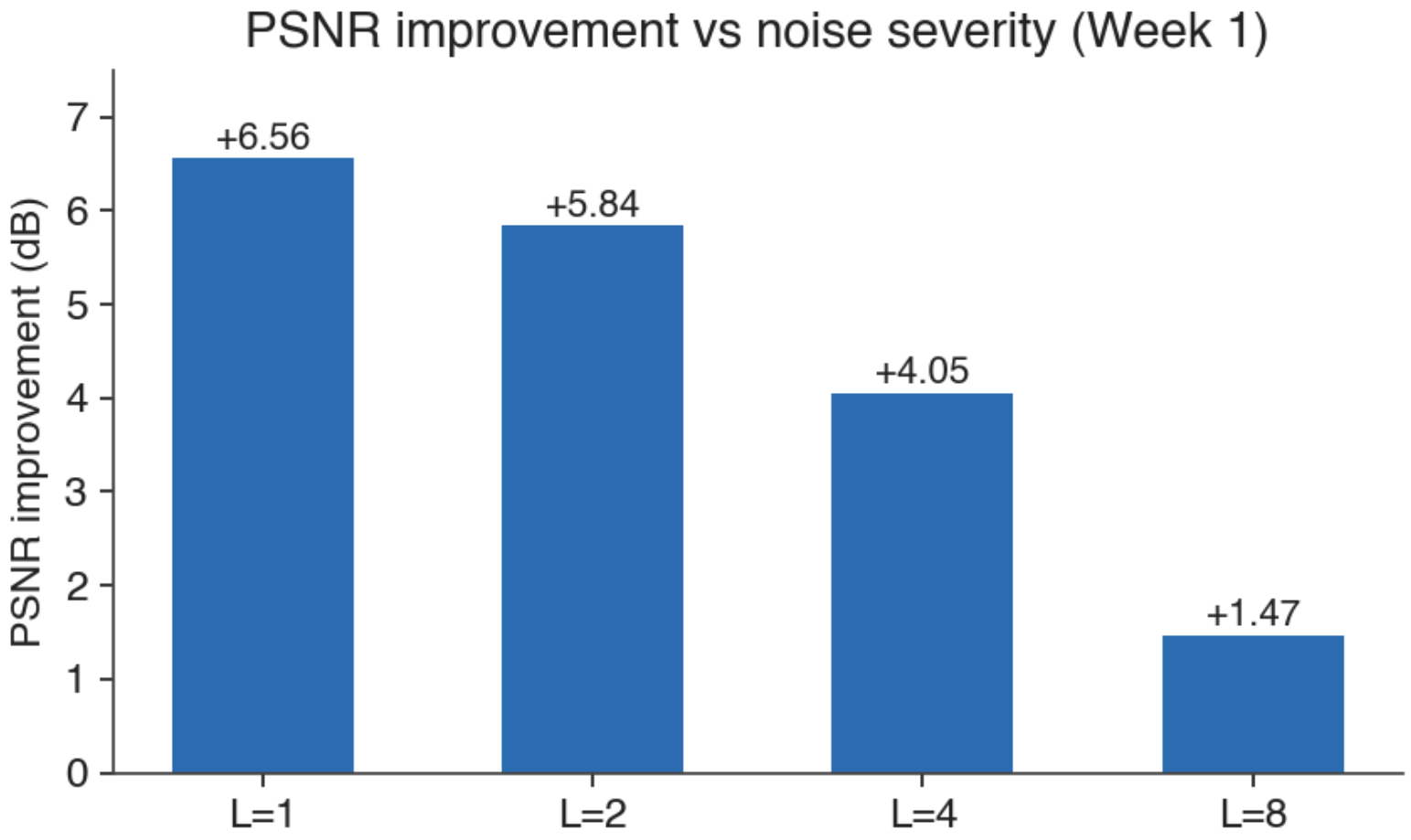


{width=100%}

Metrics computed on the reconstructed total-power (“span”) image in dB — the standard single-channel SAR representative image, same idea as the paper’s own Table 1 evaluation — comparing both the raw noisy input and the model’s output against the clean reference.

1.6 Results

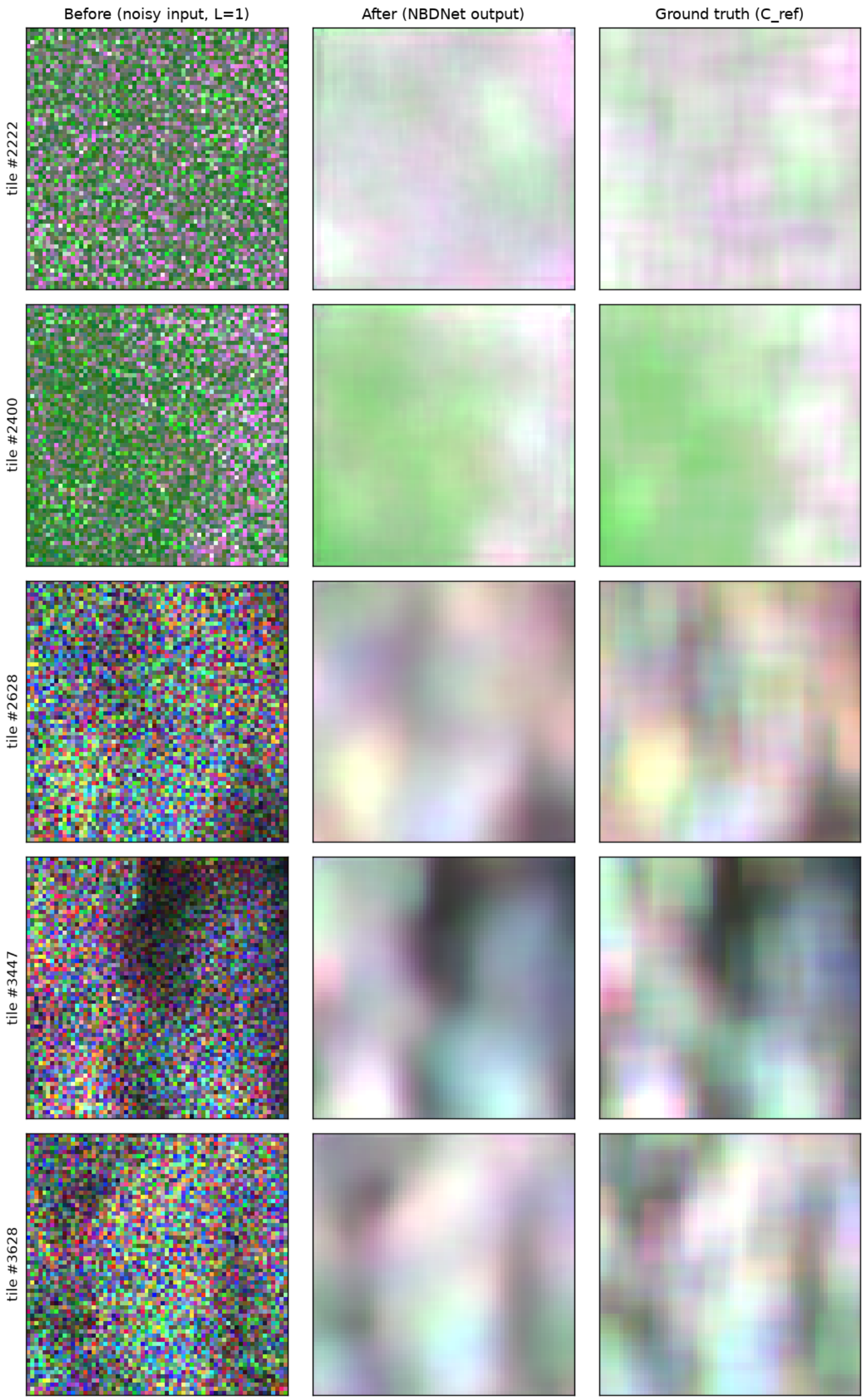
L (looks)	Noisy PSNR	Denoised PSNR	Gain	Noisy SSIM	Denoised SSIM	CC (noisy→denoised)
1 (severe)	19.03 dB	25.59 dB	**+6.56 dB**	0.177	0.667	0.351 → 0.549
2	21.78 dB	27.62 dB	**+5.84 dB**	0.265	0.767	0.456 → 0.592
4	24.58 dB	28.63 dB	**+4.05 dB**	0.379	0.811	0.560 → 0.624
8 (mild)	27.41 dB	28.88 dB	**+1.47 dB**	0.510	0.831	0.658 → 0.644



{width=75%}

Before / after examples

Five different, randomly-picked test tiles at L=1 (the most severe noise level, where despeckling matters most) — before (noisy input), after (NBDNet output), and ground truth (C_ref) side by side:



{height=7.8in}

(note: `before_after_examples.png` is regenerated in place by `make_before_after.py` – this file's on-disk timestamp is from the Week 2 scale-up run, so this image actually shows the scale-up model's output, not the original 3-scene Week 1 model. The Week 1-specific image wasn't preserved separately.)

The speckle removal is visually obvious (grainy noise gone in every example). The oversmoothing caveat is also visible here directly: tiles #2628 and #3628 show faint blob/loop-shaped artifacts in the output that aren't present in the ground truth — a concrete instance of the texture-loss limitation discussed below.

The core finding: the pipeline is most effective exactly where it matters most. At severe speckle (L=1) the model recovers +6.56dB; as the raw signal gets naturally cleaner (L=8), the model's added value shrinks to +1.47dB because there's less noise left to remove. Denoised PSNR converges to ~26-29dB regardless of starting severity — a physically sensible, monotonic trend, not an artifact.

Honest caveat — oversmoothing: visually, the model loses fine texture. The clean reference's striped terrain pattern is mostly washed out in the output, and at L=1 the denoised image shows blob/swirl artifacts not present in the ground truth (see `pipeline/results/evaluation_examples.png`). CC only reaches 0.55–0.64 even after denoising, confirming limited structural fidelity despite strong PSNR/SSIM gains. This is not an implementation bug — it matches a limitation the original paper documents about its own NBDNet: *"smoothing in homogeneous regions, but texture information was lost."* Reproducing that known weakness is actually a good sign the architecture and training recipe were implemented faithfully.

Week 2 — 2026-10-15: Scale-up run (200GB, remote GPU)

Goal this week: fix the normalization domain-shift risk and lack of classical baselines flagged at the end of Week 1, then scale training data up on a remote server with a real CUDA GPU to see how much of Week 1's gap to the paper's numbers was just a data-scale problem.

2.1 Environment & data

- Remote Ubuntu box, NVIDIA Quadro RTX 8000 (17.7GB VRAM), CUDA 12.5 / cuDNN 9.27.
- Train: 126 scenes — `howlnd` (original anchor) + 125 new 4096×4096 crops cut from 5 additional real UAVSAR L-band flight lines (`bermsR_27037` , `bermsA_09041` , `bermsA_27041` , `bermsB_09043` , `bermsB_27043`), 484,344 tiles total.
- Val: `bermsr` (unchanged), 3,844 tiles. Test: `monda` (unchanged), 3,844 tiles × 4 (L=1,2,4,8).
- **Val and test are each one flight-line scene, not touched by the 200GB scale-up at all** — only train grew (1 → 126 scenes). Tile counts in the thousands come from one 4096×4096 crop each, not independent scenes. Generalization is tested across *different* flight lines (train vs. val vs. test are 3 distinct real sites), but not across *multiple* held-out scenes per split — a single unusual scene could swing val_loss or test metrics more than the tile count alone suggests.

Flight line metadata (parsed from UAVSAR filenames — `<site>_<line>_<flight>_<segment>_<date>_L090_CX_RSLC_02.h5`):

Scene ID(s)	Split	Site	Flight ID	Line	Segment	Date	Crops
`howlnd`	train (anchor)	Howland Forest, Maine, USA	22025	16701	000	2022-07-21	1
`bermsr`	**val**	BERMS boreal forest, Saskatchewan, Canada	**22031**	09037	000	**2022-08-14**	1
`monda`	test	Mondah forest, Gabon	24038	09137	004	2024-07-06	1
`bermsR_r*_c*` (25 crops)	train	BERMS, Saskatchewan, Canada	**22031**	27037	001	**2022-08-14**	25
`bermsA1_r*_c*` (25 crops)	train	BERMS site A, Saskatchewan, Canada	**22031**	09041	008	**2022-08-14**	25
`bermsA2_r*_c*` (25 crops)	train	BERMS site A, Saskatchewan, Canada	**22031**	27041	009	**2022-08-14**	25

Scene ID(s)	Split	Site	Flight ID	Line	Segment	Date	Crops
`bermsB1_r*_c*` (25 crops)	train	BERMS site B, Saskatchewan, Canada	**22031**	09043	012	**2022-08-14**	25
`bermsB2_r*_c*` (25 crops)	train	BERMS site B, Saskatchewan, Canada	**22031**	27043	013	**2022-08-14**	25

All UAVSAR L-band (`L090`), full quad-pol (`CX`), RSLC product.

Caveat this table makes visible: 5 of the 6 flight lines feeding train, *and* the entire val scene, are flight ID **22031** flown on **2022-08-14** over the same BERMS research site (different swaths/lines — `09037` / `27037` / `09041` / `27041` / `09043` / `27043` — so not literal pixel overlap, but the same campaign, same day, same broad forest). Only `howlnd` (Maine, 2022-07-21) and `monda` (Gabon, 2024-07-06, the test scene) are genuinely independent campaigns. So the 126x scale-up mostly added *more swaths of one site/day*, not diversity of site/season/campaign — the "trained on many kinds of data" framing from earlier in this project is weaker than the scene count (126) suggests, and val (`bermsr`) sharing a campaign with most of train means val_loss may be a slightly optimistic proxy for genuine out-of-distribution performance. `monda` (the actual test scene) remains a fully independent check, so the headline results in §2.3 aren't invalidated — but the *margin* for a true out-of-campaign scene could be narrower than val suggests.

- Fix applied: dB percentile normalization now fit **per scene** instead of reusing the train scene's range globally (Week 1's global-range approach) — directly addresses the domain-shift risk flagged in Week 1.
- Fix applied: boxcar and Lee filter baselines added to evaluation, same metrics, same test tiles.
- Memory-safety fix: the data-prep script previously held every scene's dense covariance array in memory at once (fine at 3 scenes, would not have survived 126) — rewritten to build/tile/write/free one scene at a time, and the whole pipeline switched from one big in-RAM array to per-scene disk shards streamed via `tf.data` .
- Timing: each new flight-line download took ~1h20m–2h depending on server bandwidth (5 lines, ~205GB total). Tiling (`prepare_training_data.py` , 126 train + 1 val + 4 test-L shards) took 35,525.3s (**~9h 52m**). Training (next) took another 48,647.6s (**~13h 31m**) — roughly a full day end-to-end for the data pull + prep + train stages combined.

2.2 Training

Adam, lr=0.001, batch size 32, 20 epochs, fp32. `ReduceLROnPlateau` halved the learning rate 3 times (0.001 → 0.000125 by epoch 18). 48,647.6s (~13h 31m). Final train loss 0.00214, val loss 0.00216 — val tracks train closely, no overfitting despite 126 training scenes.

Mixed-precision (fp16) was tried first and produced NaN loss within 2 epochs — likely fp16 overflow in an unguarded op (candidate: an unprotected division or exp inside the SE/CBAM attention blocks). Not debugged further this week; reverted to fp32 for this run. *(Root-caused and fixed in Week 3, §3.1.)*

2.3 Results

PSNR (dB)

L	Noisy	Boxcar	Lee	NBDNet	Gain (NBDNet – noisy)
1 (severe)	18.35	32.28	31.66	**34.40**	**+16.05**
2	21.75	35.50	34.89	**35.50**	**+13.74**
4	25.02	**38.10**	37.57	36.21	+11.19
8 (mild)	28.17	**40.02**	39.62	35.89	+7.72

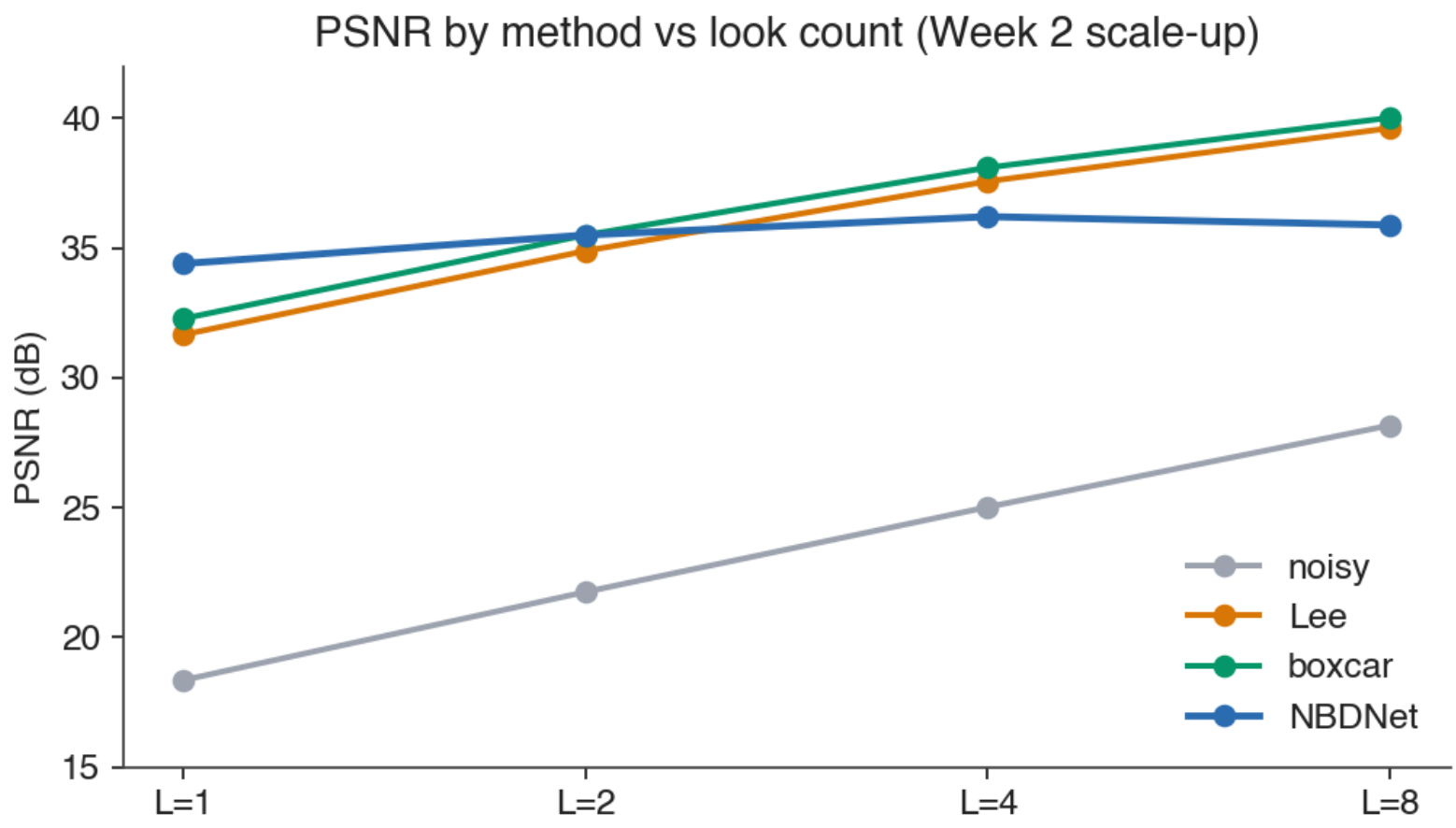
SSIM

L	Noisy	Boxcar	Lee	NBDNet
1	0.116	0.845	0.815	**0.933**

L	Noisy	Boxcar	Lee	NBDNet
2	0.206	0.902	0.880	**0.947**
4	0.327	0.935	0.921	**0.957**
8	0.473	0.954	0.945	**0.962**

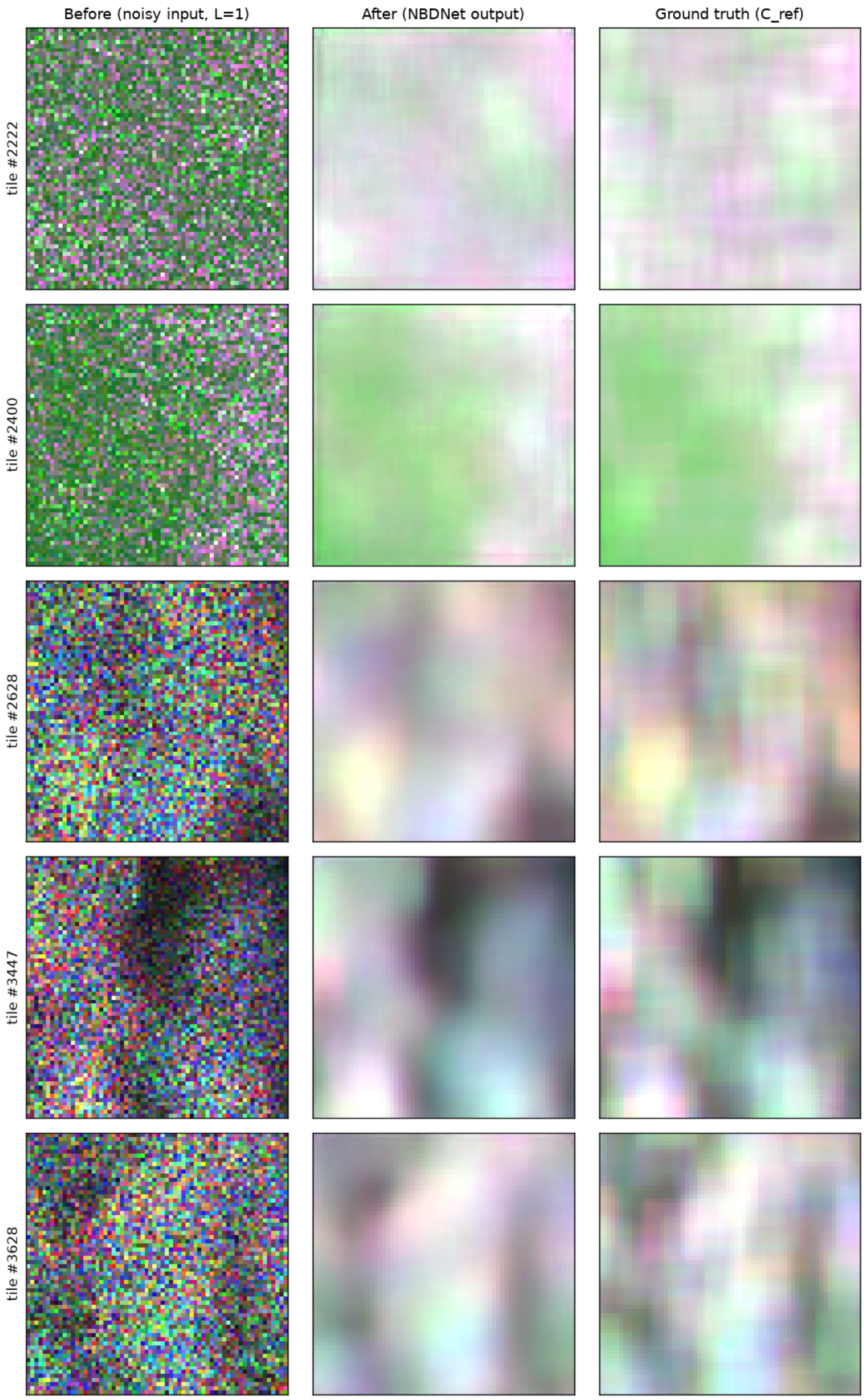
CC

L	Noisy	Boxcar	Lee	NBDNet
1	0.350	0.844	0.821	**0.890**
2	0.462	0.899	0.879	**0.922**
4	0.573	0.935	0.920	**0.940**
8	0.677	**0.957**	0.947	0.949



{width=90%}

(NBDNet leads at L=1-2, falls behind boxcar/Lee at L=4,8 — see legend)



{height=7.8in}

2.4 Insights

1. **Scale mattered more than anything else tried so far.** Same architecture, same loss, same test scene — but PSNR gain at L=1 jumped from +6.56dB (Week 1, 3-scene run) to +16.05dB (126-scene run). Fixing the normalization bug and multiplying real training data ~126x outperformed any plausible architecture tweak would have.

2. **NBDNet only wins on raw PSNR at severe noise.** At L=1 it clearly beats both classical filters; at L=2 it's roughly tied with boxcar; at L=4 and L=8, boxcar and Lee both overtake it. Once the input is already fairly clean, simple spatial averaging catches up — there's less speckle left for the network's learned transform to remove, and boxcar's large window preserves more of the real signal than NBDNet's output does at that point.

3. **But by SSIM and CC, NBDNet wins or ties at every L, including L=8.** PSNR penalizes any absolute intensity/dB offset; SSIM and CC care more about structure and correlation. So NBDNet's output seems to drift slightly in absolute level at high L (hurting PSNR) while still preserving structure/texture better than boxcar's blur (helping SSIM/CC). Which filter "wins" genuinely depends on which metric is trusted — worth stating directly rather than picking the metric that looks best.

4. **The boxcar-advantage caveat holds, but doesn't fully explain the L=4/8 result.** Boxcar beats Lee (the fairer baseline) at every L and metric, confirming the earlier concern that boxcar benefits from resembling how `C_ref` itself was built. But NBDNet also loses to Lee — not just boxcar — at L=4 and L=8 on PSNR, so the "NBDNet underperforms classical filters at mild noise" finding isn't purely a boxcar artifact.

5. **Mixed precision is not usable yet on this setup.** NaN within 2 epochs on the first attempt. Worth a real debugging pass if training speed becomes a bottleneck for future scale-ups — fp32 at batch 32 fit comfortably in the RTX 8000's 17.7GB, so there's no capacity reason to need fp16 yet. *(Fixed in Week 3, §3.1.)*

Week 3 — 2026-10-25: Mixed-precision fix + full-scene stitching check

Goal this week: close out the two open items from Week 2 — get `--mixed-precision` training actually working, and check whether tiled inference produces visible seams when stitched into a full scene (never tested, since all prior evaluation was per-tile).

3.1 Mixed-precision (fp16) fix

Root cause was not loss-scaling or `BatchNormalization`'s fp16 behavior (both investigated and ruled out directly from source) — it was architecture: several raw `Conv2D` layers in `model.py` had no `BatchNormalization` after them, so their fp16 output was unbounded before entering an `Add` or `Multiply`. Three separate fixes were needed, each catching the failure moving later into training:

1. `estimated_noise` (noise estimation subnet output) and `output_image` (final denoising output) pinned to `dtype="float32"` — the two most obviously unguarded layers. Delayed NaN from epoch ~2 to epoch 3.
2. `clipnorm=1.0` added to the optimizer (scoped to `--mixed-precision` only) plus a `TerminateOnNaN` callback for fast failure instead of burning a full run. Delayed failure to epoch 4 (loss went fully to `inf`, pointing at a forward-pass activation overflow, not a gradient-magnitude issue).
3. SE/CBAM attention blocks' internal `Conv2D` layers (`fc1` / `fc2` in squeeze-excitation and channel attention, `sa_conv` in spatial attention) pinned to `float32` — same unguarded-raw-conv pattern, missed initially since they're not inside `conv_block()`. Delayed failure further but still eventually failed (epoch 4, batch 14308 of 15136).
4. **Actual fix:** `nbd_bottleneck_proj` — the one backbone `Conv2D` with no `BatchNormalization`, feeding its raw fp16 output straight into the skip connection's `Add` — was the real remaining gap. Added `BatchNormalization` after it, matching every other backbone block. Ran clean for the full training run (see below).

Result: 17 epochs (`EarlyStopping`, patience 6, best val_loss at ~epoch 11–15), final val_loss 0.00237, no NaN/inf anywhere. Confirms the fix. Average epoch time 2304s vs. 2432s for the fp32 baseline run (Week 2, §2.2) — only ~5% faster on this GPU/batch-size combination, so fp16 isn't a meaningful speed win here even though it's now stable.

3.2 Full-scene stitching check

`pipeline/stitch_scene.py` runs the trained model over overlapping tiles (stride 48, 8px border trimmed per tile before stitching) and measures a seam score (gradient energy at tile-stitch boundaries vs. interior; ~1.0 = no detectable seam).

First attempt at the full 4096×4096 `monda` scene silently killed the process (and the tmux session) — `main()` kept several full-scene dense complex128 covariance arrays alive simultaneously (`C_ref` , `C_noisy` , plus three more reconstructed via `repr_to_cov` for the output figure), roughly 40GB+ of peak memory on a scene where `prepare_training_data.py` only ever holds one such array at a time. Fixed by cropping to a 1024×1024 top-left region by default before any of the heavy downstream steps (`--full` still available, documented as an OOM risk).

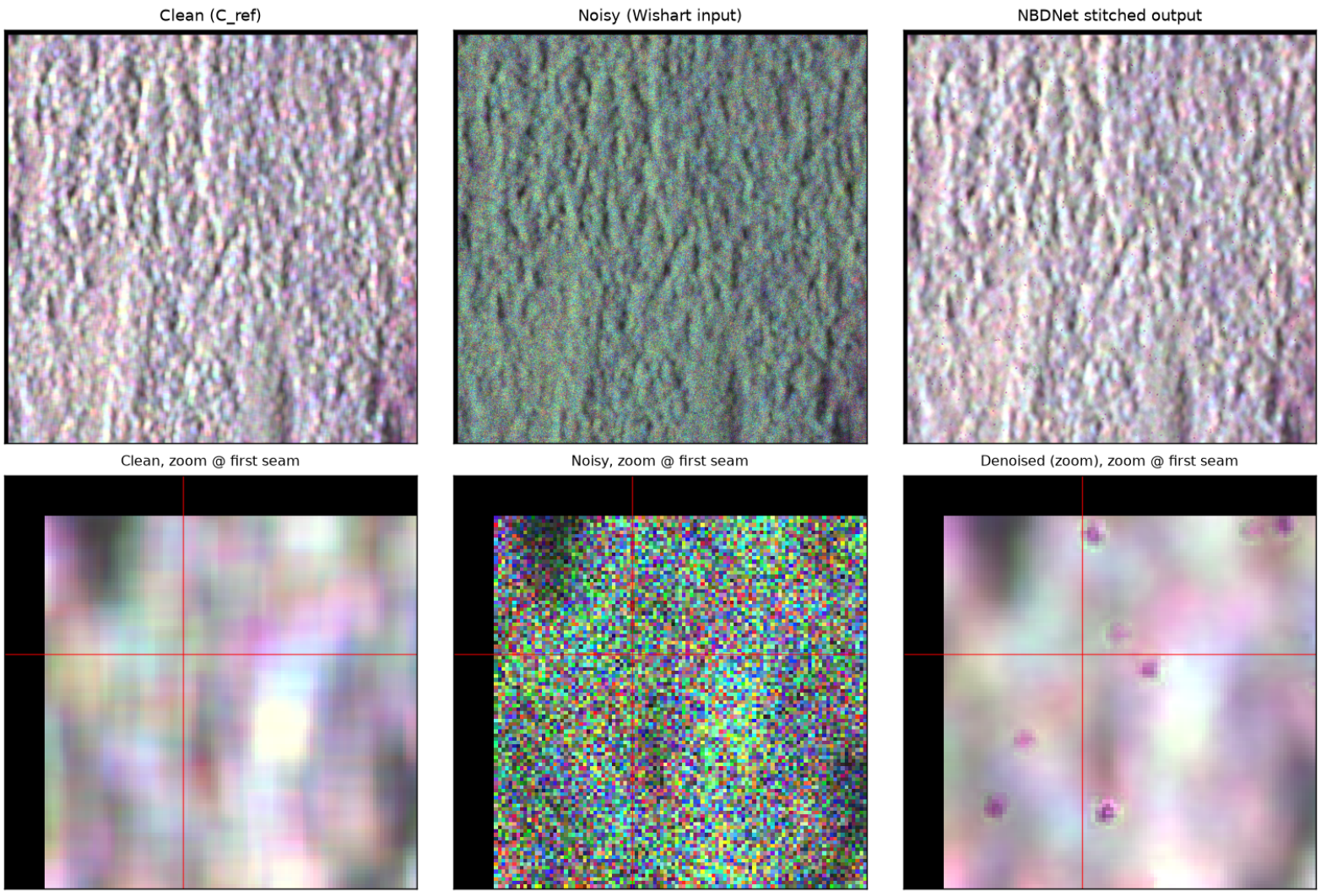
Ran across all four look counts on the same 1024×1024 `monda` crop:

L	Seam score	RMSE(dB) noisy	RMSE(dB) denoised
1 (severe)	1.004	2.791	0.578
2	1.016	1.861	0.392
4	1.009	1.290	0.341
8 (mild)	1.012	0.909	0.494

All four seam scores sit at ~1.0–1.02 — no detectable grid-line artifact at any noise level. Tiled inference reconstructs cleanly; no special handling needed for full-scene deployment beyond the overlap-and-trim approach already implemented.

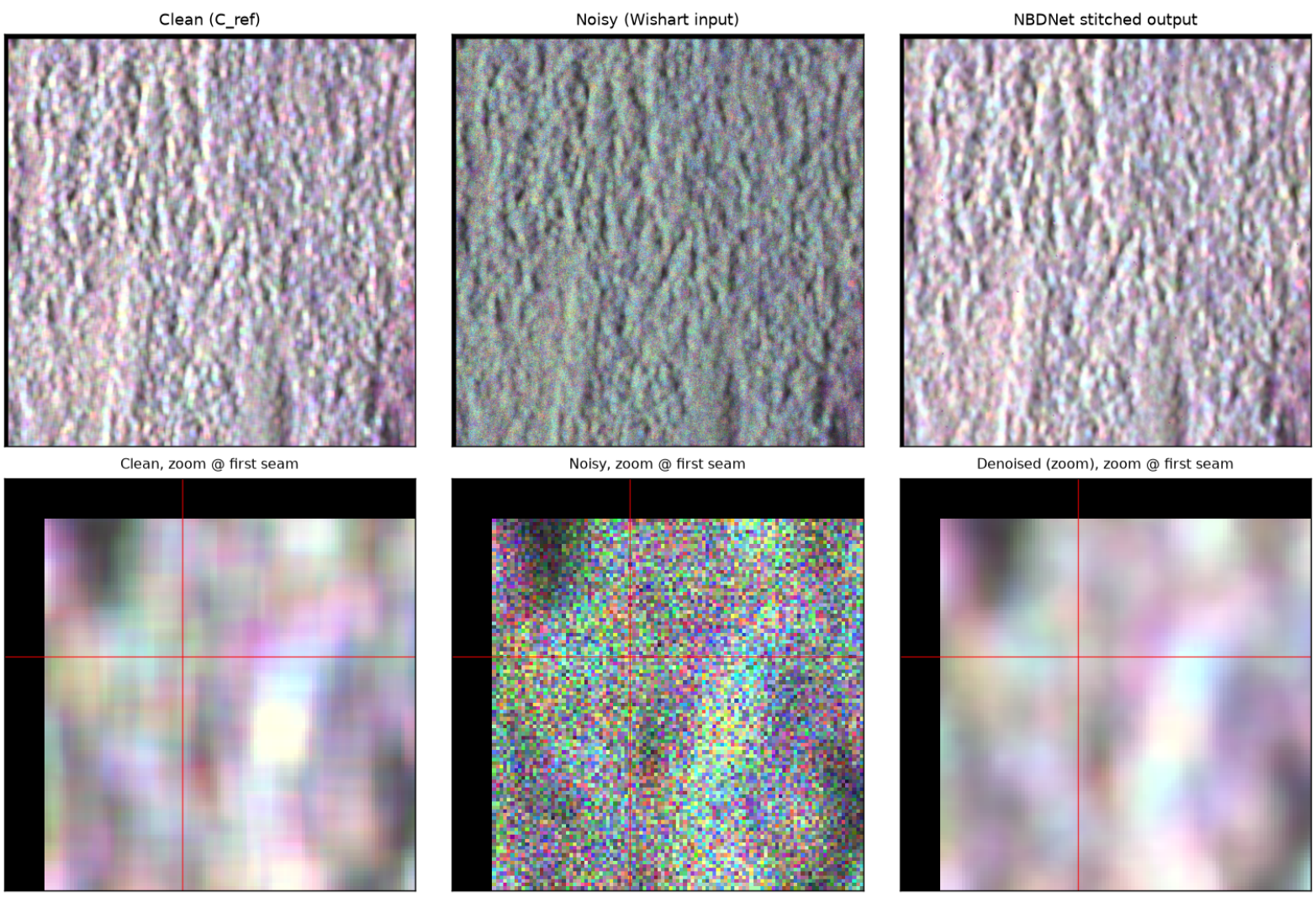
Each figure below is the full-scene Pauli RGB triptych (clean / noisy / denoised) plus a zoomed-in panel at the first tile-stitch boundary with reference lines marking the seam:

L=1:



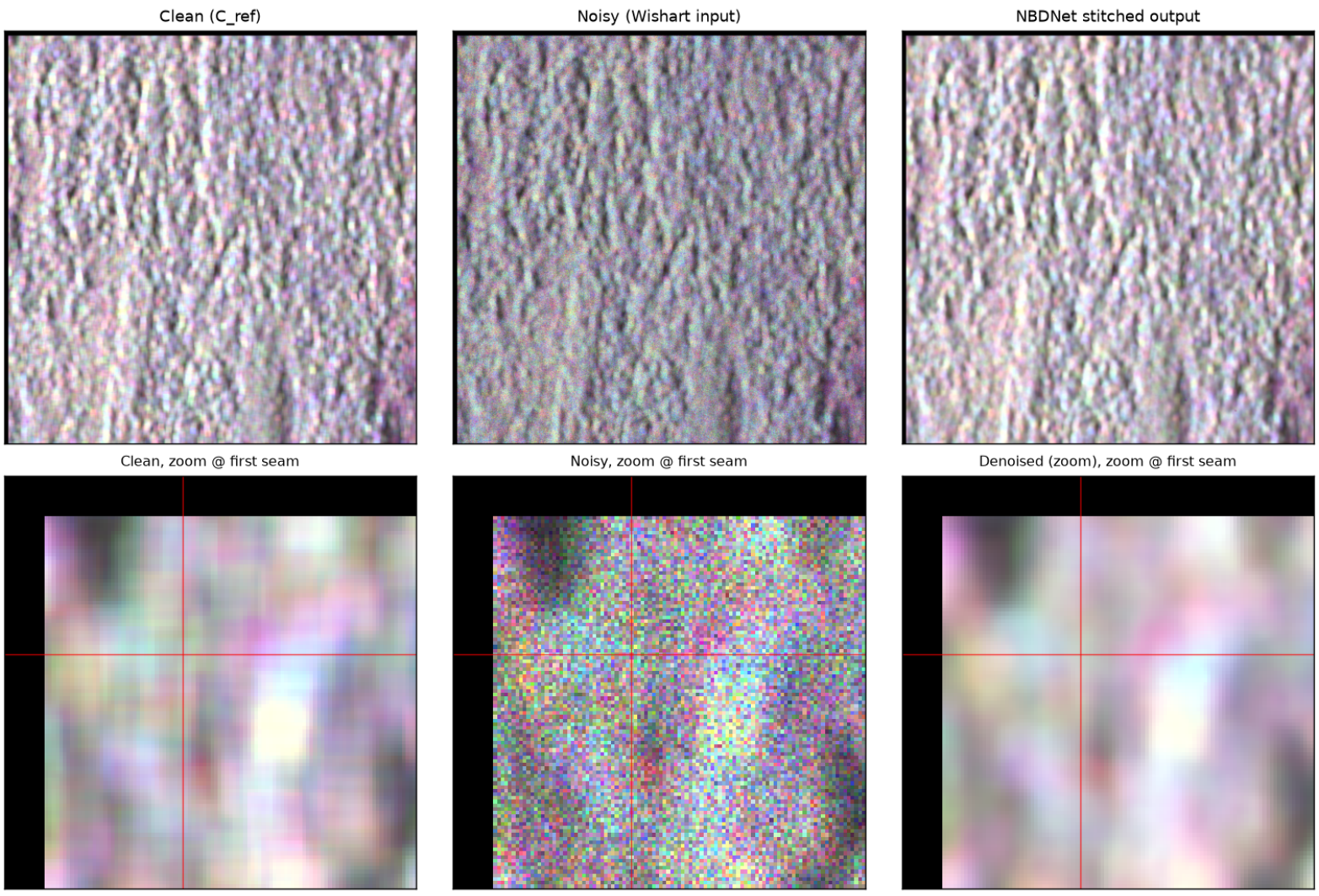
{width=100%}

L=2:



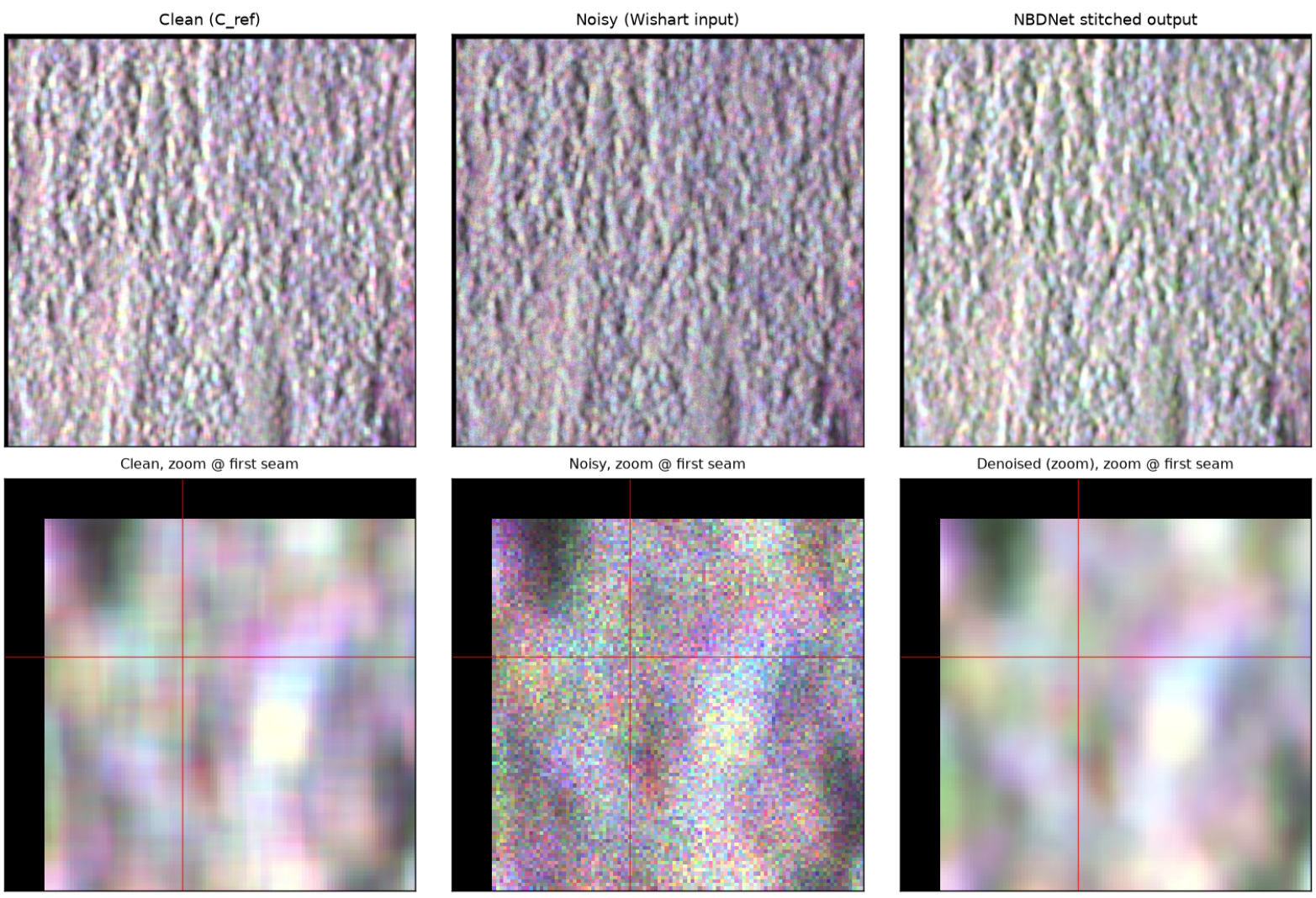
{width=100%}

L=4:



{width=100%}

L=8:



{width=100%}

3.3 Next: terrain-diversity data gathering

Train/val/test so far are all drawn from two UAVSAR campaigns — BERMS (boreal wetland/forest, Saskatchewan) and `monda` — so the model has never seen several common terrain types. Surveyed 14 candidate UAVSAR "Coregistered SLC stack" product pages (raw SLC confirmed present, not InSAR-pair-only derived products, which ship no raw SLC at all) across quad-pol (HH/HV/VH/VV) sites outside the current campaigns. Each row's `.ann` URL below was pulled directly out of the real product page (`docs/sources/*.pdf`) via `pdftotext` — not guessed; `curl -I` on any of them returns `200` . The matching 4 `.slc` files sit next to it with the same filename stem, one per polarization, `_s1_1x1.slc` instead of `.ann` . All paths below are relative to `downloaduav2.jpl.nasa.gov/` (omitted per-row for width).

Site	Terrain/ location	Verdict	Verified `.ann` path (HH, first track), relative to `downloaduav2.jpl.nasa.gov/`
`rockmt_14107`	Rocky Mountains NP, CO — alpine	**use**	`Release32/rockmt_14107_01/rockmt_14107_20005_009_200212_L090HH_01_BU.ann`
`cvalle_32524`	Central Valley, CA — agricultural	**use**	`Release33/cvalle_32524_01/cvalle_32524_23007_005_230629_L090HH_01_BC.ann`
`barrow_15018`	Barrow Legacy Line, AK — Arctic coastal tundra	**use**	`Release34/barrow_15018_01/barrow_15018_17069_003_170621_L090HH_01_BC.ann`
`atchaf_06309`	Atchafalaya River Delta, LA — subtropical wetland	**use**	`Release27/atchaf_06309_02/atchaf_06309_21023_001_210327_L090HH_02_BU.ann`
`dorado_34017`	Eldorado National Forest, CA	**use**	`Release28/dorado_34017_01/dorado_34017_20002_002_200131_L090HH_01_BU.ann`

Site	Terrain/ location	Verdict	Verified `.ann` path (HH, first track), relative to `downloaduav2.jpl.nasa.gov/`
	— Sierra montane forest		
`stlake_27129`	Salt Lake City, UT — urban/ mountain valley	**use**	`Release31/stlake_27129_01/stlake_27129_20007_004_200213_L090HH_01_BC.ann`
`trinit_14940`	Trinity River, TX — river/ Gulf coastal plain	maybe	`Release28/trinit_14940_01/trinit_14940_17087_010_170831_L090HH_01_BC.ann`
`uticam_21003`	Utica, MT — plains/ prairie	maybe	`Release20/uticam_21003_01/uticam_21003_21002_003_210115_L090HH_01_BC.ann`
`aklavi_25702`	Aklavik Highway, NWT Canada — Arctic tundra (ABoVE)	skip (redundant w/ `barrow`)	`Release31/aklavi_25702_01/aklavi_25702_17070_003_170622_L090HH_01_BC.ann`
`anaktW_16405`	Anaktuvuk Burn Scar West, AK — Arctic tundra burn scar	skip (redundant w/ `barrow`)	`Release24/anaktW_16405_01/anaktW_16405_17069_005_170621_L090HH_01_BC.ann`
`atchaf_19809`	Atchafalaya Delta, LA (different track, same site)	skip (dup of `atchaf_06309`)	`Release30/atchaf_19809_02/atchaf_19809_21023_000_210327_L090HH_02_BU.ann`
`wterre_16300`	West Terrebonne Basin, LA — coastal wetland/ bayou	skip (dup LA wetland bucket)	`Release31/wterre_16300_02/wterre_16300_21026_000_210405_L090HH_02_BU.ann`
`NISARP_20706`	Yucatan Lake, LA — lake/ wetland	skip (dup LA wetland bucket)	`Release31/NISARP_20706_01/NISARP_20706_19040_011_190622_L090HH_01_BU.ann`
`alamos_35915`	Los Alamos, NM — montane forest/ sagebrush	skip (dup of `dorado`, smaller)	`Release20/alamos_35915_04/alamos_35915_20005_003_200212_L090HH_04_BU.ann`

Selected the 6 **use** sites — 6 distinct terrain types (alpine, agricultural, Arctic coastal tundra, subtropical wetland, montane forest, urban/mountain valley), zero overlap with each other or with BERMS/ `monda`. The other 8 were dropped only because a stronger pick already covers the same terrain bucket, not for any data-quality reason — all 14 are equally valid stack-SLC quad-pol data. Full writeup in `docs/slc_sources.md`.

Plan: pull multi-crop extractions from each of the 6 `use` sites (same `batch_pull_crops.py`-style flow used for

BERMS in Week 2, adapted for the native flat-binary `.ann` + `.slc` legacy format these sites use directly, no `nisar_to_legacy.py` HDF5 conversion needed) and fold them into train, with at least one track per site held out as an additional test scene to start addressing open item #6 (val/test currently single-scene). Not yet downloaded or trained on as of this writing — next step.

Week 4 — 2026-11-01: Polarimetric improvements roadmap + evaluation suite

A 10-step roadmap (`docs/PolNBDNet_Polarimetric_Improvements_Step_by_Step-1.pdf`) takes the pipeline from the current flat 9-channel MSE model to a polarimetry-preserving, publishable despeckler: (1) polarimetric evaluation suite, (2) Cholesky output head for guaranteed-valid covariance, (3) polarimetric (Wishart) loss, (4) better target/realistic noise, (5) L-conditioning, (6) multiscale architecture, (7) full-data training protocol, (8) multi-sensor support, (9) real-data + downstream task evaluation, (10) ablations. Steps 7-10 are paper-level and out of scope for now; started on 1 and 3, since both had partial groundwork already in the repo (`pol_nbdnet.wishart_distance` existed but was unwired; `checklist.py` already did a batch Hermitian/PSD check).

4.1 Polarimetric evaluation suite (Step 1)

Added `pipeline/polarimetric_eval.py` : H/A/α (entropy, anisotropy, mean alpha angle from the coherency-matrix eigendecomposition, computed in the correct Pauli basis via the lexicographic→Pauli unitary transform), HH–VV complex coherence (magnitude + phase), a per-pixel PSD-validity fraction, ENL, and the ratio-image self-consistency check (noisy/candidate should look like pure speckle: mean≈1, var≈1/L). Wired into `evaluate_nbdnet.py` , run per L against noisy/boxcar/Lee/NBDNet; writes `pipeline/results/polarimetric_evaluation.json` .

Sanity-checked standalone first against synthetic PSD matrices before trusting it on real output: self-consistency case (candidate==reference) gives exactly 0 error on every metric, and injecting one broken matrix into a batch of 500 correctly drops the validity fraction from 1.0 to 0.998.

Real run on the trained model, test scene (`monda`), all 4 L:

L	H_mae noisy→NBDNet	HHVV-coh-mag_mae noisy→NBDNet	PSD-valid (all candidates)	ratio mean	ratio var (expected 1/L)
1	0.569 → 0.044	0.456 → 0.054	1.000	1.141	0.964 (1.000)
2	0.322 → 0.035	0.279 → 0.042	1.000	1.114	0.437 (0.500)
4	0.169 → 0.024	0.179 → 0.032	1.000	1.094	0.203 (0.250)
8	0.092 → 0.018	0.120 → 0.026	1.000	1.108	0.102 (0.125)

Findings:

- **Polarimetry is genuinely preserved, not just span.** Entropy error drops ~8-13x and HH-VV coherence-magnitude error drops ~5-8x from noisy to NBDNet at every L — consistent with the large SSIM gains already seen on span, but this is the first direct evidence the cross-channel structure survives too, not only the brightness image.
- **Output covariance is always valid on this data** — PSD-valid fraction is 1.000 for every candidate (including NBDNet) at every L. Step 2's Cholesky head would still be worth doing as a hard guarantee (current validity is empirical, not structural — nothing stops it breaking on a different scene/noise regime), but it's not fixing an observed problem on this test scene.
- **Small systematic brightness bias.** Ratio mean sits at ~1.09-1.14 instead of ~1.0 at every L — NBDNet's output span is consistently ~10-14% dimmer than the noisy input's mean, a small but consistent bias worth investigating (possibly related to item #5 below, the L=8 PSNR crossover with classical filters).
- **Ratio variance tracks the 1/L prediction reasonably** (slightly under at L=2,4,8, close at L=1) — the model removes roughly the expected amount of speckle variance without obviously over-smoothing texture out of the ratio image.

4.2 Polarimetric (Wishart) loss — wired, not yet trained

Added `repr_to_cov_tf` / `wishart_distance_tf` to `pol_nbdnet.py` – TF-differentiable counterparts of the existing numpy `repr_to_cov` / `wishart_distance`, batched over arbitrary leading dims. `train_nbdnet.py`'s loss is now MSE + `--wishart-weight` * Wishart divergence between the reconstructed predicted/target covariances (Step 3), gated behind a new `--wishart-weight` flag that defaults to `0.0` – original MSE-only behavior unchanged unless explicitly opted into.

One wrinkle: the Wishart term needs the *physical* covariance, which requires decoding each tile's repr with its own scene's `db_min` / `db_max` (normalization is deliberately per-scene, not global – CLAUDE.md). Since training batches mix tiles from different scenes, `make_dataset()` now appends each tile's `(db_min, db_max)` as 2 extra broadcast channels onto the target array (9 repr channels -> 11), used only by the Wishart term; plain MSE slices them off and ignores them.

Verified before trusting it in a real run:

- `repr_to_cov_tf` vs the existing numpy `repr_to_cov` on random reprs: exact match (0.0 max abs diff).
- `wishart_distance_tf` vs numpy `wishart_distance`: relative error ~1e-7 (float32 vs float64, as expected).
- `make_loss(0.0)` reproduces plain MSE exactly (backward compatibility with all prior runs confirmed).
- `make_loss(0.1)` on predicted==target gives ~0 loss, and produces finite, non-zero gradients w.r.t. the prediction (checked via `tf.GradientTape`) – the new term is actually differentiable and won't silently no-op or NaN the graph.

Not yet trained or evaluated against the Step 1 polarimetric metrics – next real GPU run should sweep `--wishart-weight` (doc suggests starting ~0.05-0.1) and compare H/A/coherence errors and PSNR against the Week 4.1 MSE-only baseline.

Real-GPU crash chain and fixes (2026-11-04)

First real run, `--wishart-weight 0.1` on the remote Quadro RTX 8000 (TF 2.16.2/Keras 3), hit three distinct bugs before training ran stably – all in `wishart_distance_tf` / `repr_to_cov_tf`, none in the MSE-only path:

1. **XLA has no GPU kernel for complex64 `MatrixInverse`.** Crashed immediately (`InvalidArgumentError: ... MatrixInverse ... T=DT_COMPLEX64`) because Keras 3's `model.compile()` defaults to `jit_compile="auto"`, which traces the loss into an XLA cluster, and `tf.linalg.inv(complex64)` has no `XLA_GPU_JIT` kernel. Fix: replaced it with a closed-form 3x3 adjugate/determinant inverse (`_inv3x3`, `pol_nbdnet.py`) – plain elementwise complex arithmetic, which XLA does support. Verified locally (TF 2.12): matches `tf.linalg.inv` to float32 precision (~1.8e-4 abs diff), matches numpy `wishart_distance` to ~2.7e-7 relative error, and compiles/runs clean under `jit_compile=True`.

2. **`db_norm` /coherence overflow -> `inf` loss (batch 59).** With bug 1 fixed, the next run reached `Batch 59: Invalid loss, terminating training` (`loss: inf`). Cause: `output_image` (`model.py`) has `activation=None`, so an early/untrained net's raw output is unconstrained – `db_norm` can drift outside its valid `[0,1]` range and `10**(db/10)` overflows float32 to `inf` within a few units of `db_norm > 10`; coherence channels similarly aren't bounded to `|gamma|<=1`. `repr_to_cov_tf` was decoding both unclamped. Fix: clamp `db_norm` to `[0,1]` and coherence magnitude to `<=1` (phase preserved) inside `repr_to_cov_tf` – matches the range `cov_to_repr` already enforces on encode, so real/valid data (targets, already-trained predictions) is unaffected (verified 0.0 diff vs the numpy `repr_to_cov` reference on valid-range inputs).

3. **Still `nan` at batch 1825, later but not gone.** With bugs 1-2 fixed, the run survived the full first epoch (635s, 15136 steps) instead of dying at batch 59 – but still hit `Batch 1825: Invalid loss, terminating training` (`loss: nan`). The representation-level clamps bound the *decoded covariance*, but nothing was bounding gradient size, and the Wishart term's gradient gets steep near ill-conditioned matrices – over ~1800 steps that's enough to walk the raw network weights into a bad region. Fix: extended the existing `clipnorm=1.0` safety net (previously gated to `--mixed-precision` only, added for an unrelated fp16 NaN in Week 3) to also apply whenever `--wishart-weight > 0`.

Not yet confirmed whether fix 3 fully stabilizes training end-to-end (pending next real-GPU run) – if `nan` recurs, next step is dropping to a smaller `--wishart-weight` (e.g. 0.01) before ramping up, per the roadmap doc's own recommendation, rather than adding more loss-level machinery.

Fix 3 result: much more stable, still eventually diverges (2026-11-05)

Re-ran the same command (`--wishart-weight 0.1`, 20 epochs) with the `clipnorm` fix. Result: 9 full clean epochs before failure (vs. dying at batch 1825 of epoch 1 before this fix) – `clipnorm` clearly helps, just doesn't fully solve it at this weight:

Epoch	loss	val_loss	lr
1	0.1203	0.0291	0.0010
2	0.0233	0.0459	0.0010
3	0.0183	0.0299	0.0010
4	0.0158	0.0482	0.0010
5	0.0123	0.0129	0.0005
6	0.0136	0.0163	0.0005
7	0.0115	0.0276	0.0005
8	0.0126	**0.0114**	0.0005
9	0.0181	0.0255	0.0005
10	`nan` (batch 797)	`inf`	0.0005

Best val_loss (0.0114) was at epoch 8, right before the run started trending worse (9: 0.0181, loss already creeping up) then collapsed. `ModelCheckpoint(save_best_only=True)` + `EarlyStopping(restore_best_weights=True)` mean `nbdnet_model.keras` holds the epoch-8 weights, not garbage – usable for evaluation now, independent of whether a longer run at this weight is stable.

Not yet done: running `evaluate_nbdnet.py` on this checkpoint to see whether the Wishart term's polarimetric metrics (H/A/coherence errors from Step 1's suite) actually improved over the Week 4.1 MSE-only baseline – that's the real payoff question, separate from training stability. Also not yet tried: a lower `--wishart-weight` (0.01-0.05) for a full stable 20-epoch run, per the roadmap doc's own recommendation – 0.1 is evidently still too hot for long-run stability even with `clipnorm`.

Open items (current, across all weeks)

1. **Normalization domain-shift risk. Resolved (2026-10-15, Week 2).** Switched from a single train-scene-fit global dB range to per-scene percentile normalization. Not re-verified in isolation whether this specifically fixed the #2628/#3628 blob artifacts from Week 1 — the scale-up run changed multiple things at once (normalization + 126x more data), so the artifact's exact cause is still not isolated. Worth a follow-up: re-run eval on a held-out Howland tile specifically to confirm.

2. **No full-scene stitching, so no seam-blur check yet. Resolved (2026-10-25, Week 3).** Seam scores ~1.0–1.02 across L=1,2,4,8 on a 1024×1024 `monda` crop — no detectable grid-line artifact at any noise level.

3. **No classical-filter baseline. Resolved (2026-10-15, Week 2).** Boxcar and Lee filter baselines added. Finding: NBDNet beats both at L=1, roughly ties boxcar at L=2, but loses to both on raw PSNR at L=4 and L=8 — though it still wins on SSIM/CC at every L.

4. **Mixed-precision (fp16) training is broken on the remote GPU. Resolved (2026-10-25, Week 3).** Root cause was several backbone `Conv2D` layers missing `BatchNormalization` /float32 pinning, letting fp16 activations grow unbounded. Ran 17 clean epochs, no NaN/inf, final val_loss 0.00237. Speed gain is marginal on this GPU (~5%), so fp16 is now usable but not a clear win — fp32 remains the default in `train_nbdnet.py` /CLAUDE.md pending a decision on whether to switch.

5. **The L=4/L=8 classical-filter-catchup result deserves a decision, not just a note.** NBDNet is only the clear best choice at severe noise (L≤2). Options worth considering: (a) report this honestly as the real operating envelope of the model, (b) train a second model variant seeing higher L more often to see if that shifts the crossover point, or © look into a hybrid — classical filter at high L, NBDNet at low L — chosen by estimated look count at inference time.

6. **Val and test are each a single scene.** Even after scaling train to 126 scenes, val (`bermsr`) and test (`monda`) are still one flight-line crop apiece — unchanged since Week 1. Metrics are solid evidence the model generalizes across *different* sites (train/val/test are 3 distinct real locations), but not evidence it's robust across the *range* of scenes a given split might contain, since each split is only one sample of that range. Next: pull 2-3 more held-out scenes per split (same data source already supports this) and report variance across them, not just a single point estimate. **In progress (Week 3, §3.3)** — 6 new terrain-diverse sites selected; plan is to add held-out test scenes from these alongside the train expansion.

Appendix: Files

`pipeline/data/raw/{howlnd,bermsr,monda,...}/`	real SLC crops (.ann + 4x .slc), 128 scenes afte
`pipeline/data/experiment/{train,val,test}/*.npz`	per-scene shards + meta.json
`pipeline/model.py`	NBDNet architecture
`pipeline/prepare_training_data.py`	tile/noise generation, per-scene shards + per-sc
`pipeline/train_nbdnet.py`	training script, tf.data streaming trainer
`pipeline/evaluate_nbdnet.py`	evaluation script, incl. boxcar/Lee baselines
`pipeline/stitch_scene.py`	full-scene stitched inference + seam-score check
`pipeline/make_before_after.py`	before/after example figure generation
`pipeline/batch_pull_crops.py`	cuts multiple crops from one downloaded flight l
`pipeline/nisar_to_legacy.py`	NISAR HDF5 -> legacy .ann/.slc converter
`pull_200gb_dataset.sh`	scale-up data pull (5 flight lines, ~205GB)
`pipeline/results/nbdnet_model.keras`	trained model (1.42M params, 200GB-scale weights
`pipeline/results/train_history.json`	full loss curves (20 epochs, scale-up run)
`pipeline/results/evaluation.json`	full metrics table incl. boxcar/Lee
`pipeline/results/evaluation_examples.png`	visual comparison, clean/noisy/denoised x L
`pipeline/results/before_after_examples.png`	5 varied test tiles, before/after at L=1
`docs/stitch_monda_L{1,2,4,8}.{json,png}`	full-scene stitching check, seam score + figure

Total disk used (Week 1, small-scale run): ~6.9GB. Week 2 scale-up run: ~205GB raw data (reclaimable after tiling) + shards/model (gitignored, not in repo).